11/10/2025

Parte B - Práctica de Procesos e Introducción a Memoria

1. (3.5pts) Suponga que se tiene el siguiente lote de procesos a ser ejecutados. Dado el Algoritmo Round Robin con Quantum de 3

JOB	Inst. Llegada	CPU	E/S (recur,inst,dur)
1	0	11	(R1, 5, 3)
2	2	9	(R2, 7, 2)
3	3	12	

a) (2.5pts) Realice el diagrama de Gantt correspondiente al lote

b) (1pt) Calcule el TPR y el TPE (incluir los cálculos realizados para llegar los valores)

SJF: Short Job First
FCFS: First Come First Served → FIFO
SRTF: Shortest Remaining Time First

Pide E/S en R1 x3 U

PROCESO	LLEGADA	CPU	PRIORIDAD	I/O(Rec,Inst,Dur)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	TR	TE
P1	0	11		(R1,5,3)	1	2	3							4	5	·	·	·				6	7	8					9	10	11				
P2	2	9		(R2,7,2)				1	2	3						4	5	6							7	·	·					8	9		
P3	3	12		—							1	2	3						4	5	6					7	8	9						10	11	12
Cola:	T0 T2 T3			T6	T9 T11 T14			T12 T20 T21 T23 T24						T27 Fin	T29 Fin	T32 Fin																				

Tiempo de Retorno: TR = Fin - llegada ⇒ $TR_1 = 22-0 = 27$ $TR_2 = 29-2 = 27$ $TR_3 = 32-3 = 29$

TR Prom = $\frac{22+27+29}{2} = 27,67$

Tiempo de Espera: TE = TR - CPU - E/S ⇒ $TE_1 = 22-11-3 = 13$ $TE_2 = 23-9-2 = 16$ $TE_3 = 29-13+0 = 17$

TE promedio = $\frac{13+16+17}{3} = 15,33$

ALG: RR

29

2. (1.5pts) Dado el siguiente pseudo-código. ¿Cuántas veces se imprime en pantalla el contenido del directorio actual del proceso?

a) 1 vez b) 2 veces c) Ninguna

```
x = fork()      → duplica proceso (Padre + hijo)
execv("ls")     → 1ev vez en Padre, 2da en hijo
exit(0)         → Chau
```

3. (2.5pts) Dado un esquema de paginación, donde cada dirección hace referencia a 1 byte, con un tamaño de página de 2 KiB (KibiBytes, siendo 1 KiB=1024 bytes) y donde el frame 0 se comienza en la dirección física 0.

Dadas las siguientes primeras entradas de la tabla de páginas de un proceso, traduzca las direcciones lógicas indicadas a direcciones físicas.

(Incluir los cálculos realizados para cada inciso)

# de Página	# de Frame
0	3
1	5
2	10
3	13
4	8

a) 7250
b) 1919
c) 5000
d) 8192
e) 6143

2 Kib = 2 x 1024 bytes = 2048 bytes

num Pag: P = Dir Lógica / Tamaño Pag

Desplazamiento: d = Dir Lógica − (P · Tamaño Pag)

Dirección Física = num de Frame f x Tamaño Pag + d

a- $P = \frac{7250}{2048} = 3$ $d = 7250 - (P \cdot 2048) = 7250 - (3 \cdot 2048) = 1106$ dir Fis = $(13 \cdot 2048) + 1106 = 27.730$

b- $P = \frac{1919}{2048} = 0$ $d = 1919 - (0 \cdot 2048) = 1919$ dir Fis = $(3 \cdot 2048) + 1919 = 8063$

c- $P = \frac{5000}{2048} = 2$ $-(2 \cdot 2048) = 904$ dir Fis = $(10 \cdot 2048) + 904 = 21.384$

Lo mismo par

4. (2.5 pts) Suponiendo que se dispone de un espacio de direcciones virtuales de 32 bits, donde cada dirección referencia 1 byte y se utilizan 18 bits de la dirección para indicar página y 14 bits para indicar desplazamiento dentro de la página (en cada inciso se deben incluir los cálculos realizados para obtener el resultado)

a) ¿Cuántas páginas puede tener un proceso?
b) ¿Cuál es el tamaño de una página? (indicar el **resultado en bytes**)
c) Si se utiliza una tabla de páginas de 1 nivel y cada entrada en la tabla de páginas es de 2 bytes ¿cuál sería el tamaño máximo que podría alcanzar la misma? (indicar el resultado en bytes)
d) Si el proceso necesitará 13798 bytes para sus datos ¿cuántas páginas se requerirán para almacenarlos?
e) Si el proceso necesitará 84542 bytes para sus código ¿cuanto sería la fragmentación interna que se producirá?

a- Cant Pags = $2^{\text{bits de Pag}} = 2^{18} = 264.144$ Pags → = 256 KiPáginas

b- Tamaño Pag = $2^{\text{bits de desplaz}} = 2^{14}$ bytes = 16.384

c- Tamaño de tabla = num TOT Pags x Tamaño de cada entrada = $264.144 \text{ Pags} \cdot 2 \frac{\text{bytes}}{\text{Pags}} = 524.288$ bytes

d- Tamaño de 1 Pag me dió: 16.384 bytes → > 13.798 bytes
⇒ Solo necesito 1 Pág → $\frac{13.798}{16.384} = 0,842 = 1$ Pag

e- 1. Calculo cuantas Pags enteras necesito ocupar: Pags necesarias = $\left\lceil \frac{84.542}{16.384} \right\rceil = 5.16 \rightarrow 6$ Pags

2. Calculo el espacio TOT. asignado en RAM para esas 6 Pags: espacio asignado = 6 x 16.384 = 98.304

3. La Fragmentación interna es el espacio desperdiciado de la última página

Fragmentación interna = 98.304 − 84.542 = 13.762 bytes

27/4/2026

Parte B - Práctica de Procesos e Introducción a Memoria

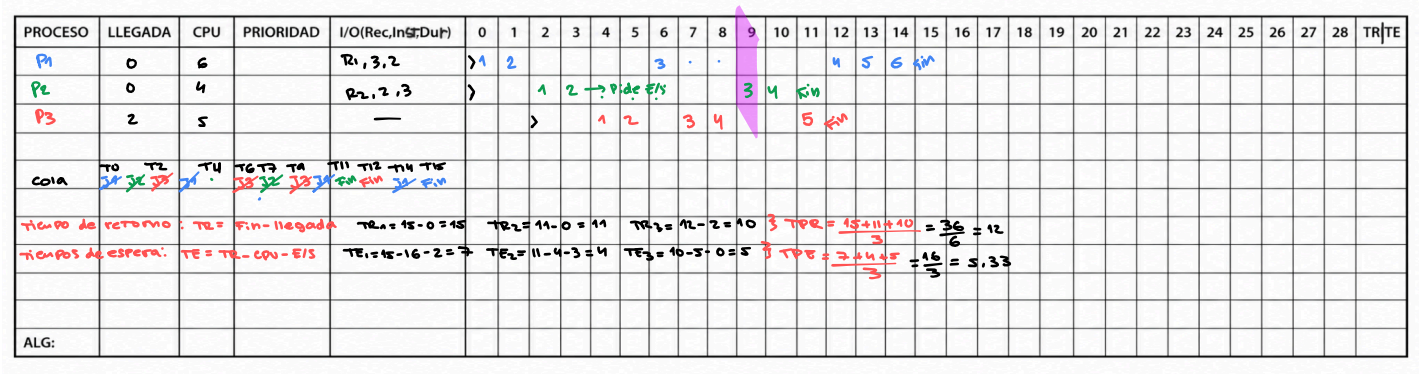
1) (3pts) Suponga que se tiene la siguiente tabla de procesos a ser ejecutados.

JOB	Inst. Llegada	CPU	E/S (recur,inst,dur)
1	0	6	(R1, 3, 2)
2	0	4	(R2, 2, 3)
3	2	5	

Dado el algoritmo: RR Q=2

a) (2pts) Realice el diagrama de Gantt

b) (1pt) Calcule el TPR y el TPE



2) (2pts) Suponga un SO que utiliza el siguiente algoritmo para la planificación de procesos:

"Se trata de un algoritmo de **tres colas** con distintos niveles de prioridad (alta, media y baja). El SO selecciona los procesos comenzando siempre desde la cola de mayor prioridad, y solo atiende una cola inferior si la superior está vacía. Cada cola es administrada por un algoritmo diferente:

- Cola alta: **RR con Quantum = 2**
- Cola media: **RR con Quantum = 6**
- Cola baja: **FCFS**

Todos los procesos nuevos ingresan a **la cola de mayor prioridad**. Si un proceso agota su quantum en la cola alta, es degradado a la cola media. Si agota su quantum en la cola media, es degradado a la cola baja. Si un proceso abandona la CPU voluntariamente (por E/S) antes de agotar su quantum, **permanece en la misma cola** al regresar."

1. ¿A qué tipo de proceso (carga de E/S o carga de CPU) beneficia el algoritmo? Justifique.
2. ¿El algoritmo puede provocar inanición de procesos? Justifique. ¿Qué mecanismo podría incorporarse para evitarla?
3. ¿Qué sucede con un proceso que realiza muchas operaciones de E/S cortas? ¿En qué cola tenderá a permanecer y por qué?

1. **Beneficia a los procesos acotados por E/S (I/O Bound).**

Justificación: Los procesos acotados por E/S realizan ráfagas de CPU cortas y se bloquean mucho al realizar operaciones de E/S. Según las reglas de este algoritmo, cuando un proceso abandona la CPU voluntariamente antes de agotar su quantum (por un bloqueo de E/S), **permanece en la misma cola al regresar**. Como los procesos acotados por E/S no suelen agotar el quantum de la cola alta ($Q = 2$), **no son degradados** y continúan ejecutando en la cola de mayor prioridad, recibiendo una respuesta inmediata cada vez que retornan de una E/S.

2. **Sí, el algoritmo PUEDE provocar inanición (*starvation*) de los procesos en las colas inferiores (media y baja).**

El sist usa un esquema de prioridad absoluta entre colas (*el SO solo atiende una cola inferior si la superior está completamente vacía*). Si constantemente están ingresando nuevos procesos o reingresando procesos desde E/S a la cola de alta prioridad, la CPU se mantendrá ocupada atendiendo la cola alta. En consecuencia, los procesos degradados a la cola media ($Q = 6$) o baja (FCFS) **pueden quedar relegados indefinidamente sin recibir tiempo de CPU**.

3. **Mecanismo para evitarla:** La incorporación de la técnica de **Envejecimiento (*Aging*)**.

Consiste en incrementar gradualmente la prioridad de los procesos a medida que transcurre cierto tiempo de espera en las colas inferiores. Si un proceso permanece en la cola baja o media demasiado tiempo sin ejecutar, el SO lo promociona a una cola de prioridad superior para garantizar que finalmente se ejecute.

3) (2.5pts) Dado un esquema donde cada dirección hace referencia a 1 byte, con páginas de **4 KiB (KibiBytes)**, donde el frame 0 se encuentra en la dirección física 0. Con las siguientes primeras entradas de la tabla de páginas de un proceso, traduzca las direcciones lógicas indicadas a direcciones físicas:

Página	Marco
0	5
1	3
2	11
3	0
4	7

a) 0:
b) 5000:
c) 7500:
d) 9000:
e) 16383:

4 KiB = 4 x 1024 bytes = 4096 Bytes
- Cada direcc referencia 1 Byte
- Frame 0 en direcc física 0

- num Pag: $P = \frac{Dir\,l}{4096}$ Direc Física = (num de marco f x 4096) + d

a - Direcc lógica: 0 → $P = \frac{0}{4096} = 0$ $d = 0 - (0 \cdot 4096) = 0$ ⇒ D. Física = 5 x 4094 + 0 = 20.480

b - D.L.: 5000 → $P = \frac{5000}{4096} = 1$ $d = 5000 - (1 \times 4096) = 904$ ⇒ D. Física = (3 x 4096) + 904 = 13.192

c - DL: 7500 → $P = \frac{7500}{4096} = 1$ $d = 7500 - (1 \times 4096) = 3.404$ ⇒ D. Física = (3 x 4096) + 3404 = 15.692

4) (2.5pts) Suponiendo que se dispone de un espacio de direcciones virtuales de 64 bits, donde cada dirección referencia 1 Byte; suponiendo además que el tamaño de página utilizado es de 4096 Bytes (4 KiB)

a) ¿Cuál sería el tamaño máximo de un proceso?
b) ¿Cuántas páginas puede tener un proceso?
c) ¿Si cada entrada en la tabla de páginas es de 8 Bytes, cuál sería el tamaño máximo que podría alcanzar la misma?
d) ¿Cuántos marcos tendrá la memoria física si disponemos de 8 GiB?
e) ¿Si el proceso necesita 10500 Bytes para sus datos, cuántas páginas se necesitan para almacenarlos?

a. $\text{Tamaño Máximo} = 2^{64} \text{ direcciones} \times 1 \text{ Byte/dirección} = 2^{64} \text{ Bytes}$

b. $\text{Bits de Desplazamiento } (d) = \log_2(4096) = 12 \text{ bits}$

$\text{Bits de Número de Página } (p) = 64 \text{ bits totales} - 12 \text{ bits desplazamiento} = 52 \text{ bits}$

$\text{Cantidad de Páginas} = 2^{\text{bits de página}} = 2^{52} \text{ páginas}$

c. $\text{Tamaño de Tabla} = \text{Número total de páginas} \times \text{Tamaño de Entrada} \rightarrow \text{Tamaño de Tabla} = 2^{52} \text{ páginas} \times 8 \text{ Bytes/página} = 2^{52} \times 2^3 \text{ Bytes} = 2^{55} \text{ Bytes}$

d. Convertimos la memoria física a Bytes: $\text{RAM Total} = 8 \text{ GiB} = 8 \times 2^{30} \text{ Bytes} = 2^3 \times 2^{30} = 2^{33} \text{ Bytes} = 8.589.934.592$

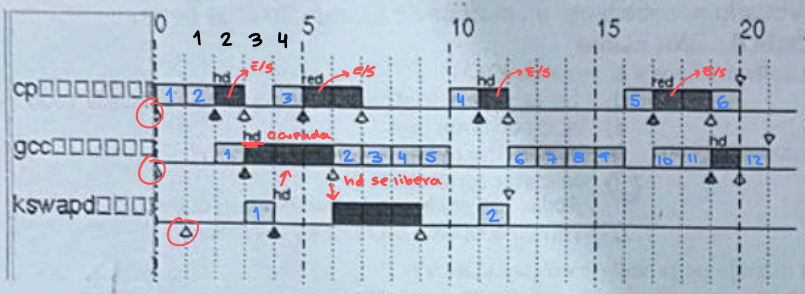
$$\text{Cantidad de Marcos} = \frac{\text{Memoria Física Total}}{\text{Tamaño de Marco}} = \frac{2^{33} \text{ Bytes}}{2^{12} \text{ Bytes}} = 2^{21} \text{ marcos} = \mathbf{2.097.152} \text{ marcos}$$

e. $\text{Páginas Necesarias} = \left\lceil \frac{10500 \text{ Bytes}}{4096 \text{ Bytes/página}} \right\rceil = \lceil 2,563 \rceil = \mathbf{3} \text{ páginas}$ (Las 2 primeras páginas alojan $4096 \times 2 = 8192 Bytes$, y la 3ª página aloja los $2308 Bytes$ restantes).

3/10/2026

Parte B - Procesos e Introducción a Memoria

1. (4 pts) Suponga que se tiene la siguiente ejecución de un lote de procesos:



- **cp:** el comando cp que está copiando un archivo del disco local a un directorio compartido por la red.
- **gcc:** el compilador de C++ compilando un programa.
- **kswapd:** proceso de apoyo para la adm. de memoria. Lleva y trae contenido desde y hacia el área de swap en disco y que acceden al disco y a la red

a) Complete la siguiente tabla con la información de la ejecución de los procesos: S/H (= Fin - llegada; = TE - CPU - E/S) b) Indique el comportamiento S/H

Proceso	Llegada	CPU	I/O (rec, instante, duración)	TR	TE	I/O Bound	CPU Bound
cp	0	6	(hd,2,1), red(3,2), hd(4,1), red(5,2)	20-0=20	20-6-6 = 8	✓	
gcc	2	12	hd(1,3), hd(11,1)	21-0=21	21-12-4 = 5		✓
ksapwd	1	2	hd(1,3)	12-1=11	11-2-3 = 6	✓	
				TPR:	TPE:	+E/S q CPU	+CPU q E/S

c) Indique cuál es el algoritmo de planificación utilizado:
FCFS SJF SRTF Round Robin Q=4 Round Robin Q=3

2. (1 pts) Dado el siguiente pseudo-código. ¿Cuántas veces se imprime en pantalla el contenido del directorio actual del proceso?
a) 1 vez **b) 2 veces** c) Ninguna

```
x = fork()
execv("ls")
exit(0)
```

(reemplaza el espacio de direcc de un proceso por la img del ejecutable; como el código original del programa ya salió, nunca se ejecuta)

3. (2.5 pts) Dado un esquema de paginación, donde cada dirección hace referencia a 1 Byte, con un tamaño de página de 2 KiB (KibiBytes, siendo 1 KiB=1024 Bytes) y donde el frame 0 se comienza en la dirección física 0.
Dadas las siguientes primeras entradas de la tabla de páginas de un proceso, traduzca las direcciones lógicas indicadas a direcciones físicas.

(Incluir los cálculos realizados para cada inciso)

# de Página	# de Frame
0	3
1	5
2	10
3	13
4	8

a) 7250
b) 1919
c) 5000
d) 8192
e) 6143

lo mismo de siempre…

4. (2.5 pts) Suponiendo que se dispone de un espacio de direcciones virtuales de 32 bits, donde cada dirección referencia 1 Byte y se utilizan 20 bits de la dirección para indicar página y 12 bits para indicar desplazamiento dentro de la página (en cada inciso se deben incluir los cálculos realizados para obtener el resultado)

a) ¿Cuántas páginas puede tener un proceso?

b) ¿Cuál es el tamaño de una página? (indicar el **resultado en Bytes**)

c) Si se utiliza una tabla de páginas de 1 nivel y cada entrada en la tabla de páginas es de 2 Bytes ¿cuál sería el tamaño máximo que podría alcanzar la misma? (indicar el resultado en Bytes)

d) Si el proceso necesitase 13798 Bytes para sus datos ¿cuántas páginas se requerirán para almacenarlos?

e) Si el proceso necesitase 84542 Bytes para sus código ¿cuanto seria la fragmentación interna que se produciría?

a. $\text{Cantidad de páginas} = 2^{\text{bits de página}} = 2^{20} \rightarrow \mathbf{2^{20}} = \mathbf{1.048.576}\ \text{páginas}$

b. $\text{Tamaño de página} = 2^{\text{bits de desplazamiento}}\ \text{Bytes} = 2^{12}\ \text{Bytes} = \mathbf{4.096}\ \text{Bytes} \quad (4\ \text{KiB})$

c. $\text{Tamaño de Tabla} = \text{Número total de páginas} \times \text{Tamaño de entrada} = 2^{20}\ \text{páginas} \times 2\ \text{Bytes/página} = \mathbf{2.097.152}\ \text{Bytes} \quad (2\ \text{MiB})$

d. $\text{Páginas necesarias} = \left\lceil \frac{13.798\ \text{Bytes}}{4.096\ \text{Bytes/página}} \right\rceil = \lceil 3,368 \rceil = \mathbf{4}\ \text{páginas}$ (Las 1eras 3 pás almacenan $3 \times 4.096 = 12.288 Bytes$, y la 4.ª pág almacena los $1.510 Bytes$ restantes).

e. $\text{Páginas necesarias} = \left\lceil \frac{84.542\ \text{Bytes}}{4.096\ \text{Bytes/página}} \right\rceil = \lceil 20,64 \rceil = \mathbf{21}\ \text{páginas}$ y $\text{Memoria asignada} = 21\ \text{páginas} \times 4.096\ \text{Bytes/página} = \mathbf{86.016}\ \text{Bytes}$

$\text{Fragmentación Interna} = \text{Memoria asignada} - \text{Bytes utilizados} = 86.016 - 84.542 = \mathbf{1.474}\ \text{Bytes}$